R1. The Phantom Stock-Out Detector

Real-world pain: A store's system says 6 units are in stock, so the item never gets reordered and never shows as out-of-stock online — but the units are actually lost, misplaced, or stolen. The shelf is empty, sales quietly go to zero, and nobody notices for weeks. This is one of retail's most expensive invisible leaks.

The problem: Given daily sales and system inventory, flag SKU-store combinations where the recorded stock is almost certainly wrong, and rank them so a store manager can physically verify only the top 20 each morning.

What we provide:

- 18 months of daily sales for ~800 SKUs across 25 stores (synthetic, with realistic seasonality, promotions, and weekday patterns)
- System-recorded on-hand inventory per SKU-store-day
- A promotion calendar and a store master (size, format, city tier)
- A hidden ground-truth label on ~3% of rows marking real phantom stock-outs (held back from teams; used by `evaluate.py`)

Assumptions students may make:

- Sales of exactly zero for N consecutive days while stock is positive is suspicious — they choose N and justify it
- Demand is roughly stable week-over-week unless a promotion or holiday is flagged
- Slow-moving SKUs legitimately have zero-sales days; the model must not drown in these false positives
- No store-level camera, RFID, or staff data exists
- A manager can verify at most 20 SKUs per store per day — the model must respect this budget

Desired output:

1. A ranked daily "go check this shelf" list: store, SKU, confidence, and a one-line reason in plain English
2. `evaluate.py` score = precision@20 per store-day (the metric that actually matters to a manager), plus recall on the hidden labels
3. A simple screen or notebook view showing the sales-vs-stock chart for any flagged SKU so the manager trusts the alert
4. An estimate of annual rupees recovered, with the arithmetic shown

Stretch: distinguish "phantom stock-out" from "misplaced on wrong shelf" from "delisted item" using different signal shapes.